

Index

NO	Description	Page. No
1	Section _A Conceptual Application	2
2	Section _B Practical Coding Task	10
3	Section _C Mini Project – Student Productivity Tracker	24
4	Section _D AI -Augmented Learning	31

GitHub Link to code file

https://github.com/Kevin-Maniar/06Aug_Kevin_SE/tree/main/Assesments

Section A – Concept Application

1. Scenario one

You are a junior developer at a fintech startup building a digital wallet app. The team is debating between Waterfall and Agile methodology. The project has frequently changing requirements and the client expects to review working features every two weeks.

Question: Which development methodology would you recommend for this project, and why? Identify two specific risks the team would face if they chose the wrong methodology for this context.

Answer: I recommend Agile methodology because agile is designed to adopt frequently changing requirements which are standard in a fast-paced industries like fintech.

Risk 1: Going with wrong methodology will lead to unmet client's requirements.

Risk 2: Late feedback and review problem which will lead to costly rework.

2. Scenario two

You are collaborating on a college project website with three teammates. Just before a client demo, one teammate pushed incomplete code directly to the main branch, breaking the layout for everyone else on the team.

Question: Describe the Git workflow your team should adopt to prevent this from happening again. Name at least two Git commands your team should use as part of this workflow and explain the role of each command.

Answers:

The workflow

1. **Protect main** — Configure the repository (e.g., on GitHub/GitLab) so no one can push directly to main. All changes must come through a pull/merge request that at least one other teammate approves.
2. **Create a feature branch** for every piece of work (a new page, a bug fix, a style change) instead of working on main.
3. **Commit work locally** on that branch as you go.
4. **Push the branch** (not main) to the remote repository.

5. **Open a pull request** comparing your feature branch to main.
6. **Teammates review** the PR — checking that it doesn't break the layout, runs correctly, etc.
7. **Merge only after approval**, ideally after testing locally or via a CI check.
8. **Delete the feature branch** once merged to keep things tidy.

Git Commands:

`git pull origin main`

- Fetches and merges the latest changes from main into your local branch, keeping you in sync and helping you catch conflicts early rather than at the last minute.

`git merge` usually done via the PR interface, or locally with

`git merge feature/navbar-fix`

- Integrates approved, tested changes from a feature branch into main in a controlled way, rather than each person pushing straight to main.

3. Scenario three

You are building an online internship application form for a college placement portal. The form must collect applicant name, email address, phone number, preferred domain (one of: Web, Data, Mobile, AI), and a resume file upload. The institution requires that no field be left empty and that the email field only accepts valid email formats — without writing any JavaScript.

Question: List the appropriate HTML input types you would use for each of the five fields. Explain how you would use HTML5 built-in validation attributes to enforce the two constraints mentioned, and why these attributes alone are not sufficient for production-level validation.

Answer:

Applicant Name – `<input type = “text”>`

Email Address - `<input type = “email”>`

Phone number - `<input type = “tel”>`

Preferred domain - `<select>` with `<option>` elements (Web,Data,Mobile,AI)

Resume Upload - `<input type = “file”>`

Enforcing the Two Constraints with HTML5 Validation Attributes

1. No field left empty → required attribute

Add required to every input, select, and file field:

Html

```
<input type="text" name="fullname" required>
```

```
<input type="email" name="email" required>
```

```
<input type="tel" name="phone" required>
```

```
<select name="domain" required>
```

```
  <option value="">-- Select Domain --</option>
```

```
  <option value="web">Web</option>
```

```
  <option value="data">Data</option>
```

```
  <option value="mobile">Mobile</option>
```

```
<option value="ai">AI</option>
</select>
<input type="file" name="resume" required accept=".pdf,.doc,.docx">
```

The browser will block form submission and show a native error message ("Please fill out this field") if any required field is left empty. Note that for `<select>`, the first `<option>` should have an empty `value=""` so it counts as "empty" and triggers the required check — otherwise the first real option would be selected by default, and the validation would never fire.

2. Valid email format → `type="email"`

Using `type="email"` alone causes the browser to enforce a basic pattern (something like `text@text.text`) automatically, without needing a pattern attribute. You can optionally tighten it further:

Html

```
<input type="email" name="email" required
      pattern="[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$">
```

Why HTML5 Attributes Alone Are Not Sufficient for Production

1. **Client-side validation can be bypassed.** A user can disable JavaScript... actually more importantly, they can disable HTML validation entirely by manipulating the DOM in browser dev tools, submitting the form via a raw HTTP request (e.g., using curl, Postman, or a script), or simply removing the `required/pattern` attributes through the browser inspector before submitting. Since validation is enforced by the browser, not the server, anyone bypassing the browser bypasses the check entirely.
2. **It doesn't verify real-world validity, only format.** `type="email"` confirms the string *looks like* an email (`x@y.z`) but doesn't confirm the email address actually exists or belongs to the applicant. Same with phone numbers — `type="tel"` doesn't verify the number is a real, reachable number.
3. **No protection against malicious input.** HTML validation doesn't sanitize input against SQL injection, XSS payloads, or oversized/malformed file uploads. A resume "file" could be a disguised executable rather than an actual PDF/DOC — `accept` is just a UI filter/hint and is trivially bypassed.

4. **No server-side guarantee of data integrity.** Since form data ultimately gets processed and stored (e.g., in a database), the backend must independently re-validate every field — checking required fields aren't empty, re-validating the email format server-side (e.g., with a regex or library), verifying file type/size by inspecting the actual file content (MIME type/magic bytes) rather than trusting the extension, and enforcing all business rules — because the server can never trust that data arriving in a request has passed through the HTML form's constraints at all.

In short: HTML5 validation attributes are a good *first line of defense* for user experience (immediate feedback, fewer accidental errors, no page reloads needed) — but because they run entirely in the client's browser, which the client fully controls, they must always be backed up by server-side validation for any production system that stores or processes sensitive user data.

4. Scenario 4

You are designing a news portal that displays articles in a 3-column grid on desktop screens and in a single-column layout on mobile devices. Your team wants to use Bootstrap's grid system to achieve this without writing any custom media query CSS.

Question: Explain how Bootstrap's grid class system works to produce this responsive layout. Write the Bootstrap class combination you would apply to the article column div and explain what each class in the combination controls.

Answers:

How Bootstrap's Grid System Works

Bootstrap's grid is built on a **12-column layout** using a system of containers, rows, and columns:

- **.container** (or **.container-fluid**) wraps the whole grid and provides responsive max-widths and padding.
- **.row** creates a horizontal flex wrapper for columns (it applies negative margins to offset the column gutters).
- **.col-*** classes are placed inside a row and each takes up a portion of the 12 available column units.

The responsiveness comes from Bootstrap's **breakpoint-specific column classes**. Instead of writing your own media queries, you use classes like **col-{breakpoint}-{number}**, where the breakpoint (sm, md, lg, xl, xxl) tells Bootstrap "apply this column width starting at this screen size and up." Bootstrap has already written the media queries for these breakpoints internally in its CSS — you just apply the right class names and it handles the rest via its **mobile-first cascading system**: a class with no breakpoint prefix (like **col-12**) applies to *all* screen sizes by default, and can be overridden at larger breakpoints.

Class Combination for the Article Column

html

```
<div class="row">  
  
  <div class="col-12 col-md-4">  
  
    <!-- article card content -->  
  
  </div>
```

```
<div class="col-12 col-md-4">  
  
  <!-- article card content -->  
  
</div>  
  
<div class="col-12 col-md-4">  
  
  <!-- article card content -->  
  
</div>  
  
</div>
```

What Each Class Controls

Class	Role
row	Wraps the article columns in a flex container, applying the correct gutters/negative margins so columns align properly
col-12	On the smallest screens (mobile, no breakpoint prefix — applies as the default/base for all sizes unless overridden), each article takes up all 12 of the 12 grid columns → full width → effectively a single-column layout
col-md-4	Starting at the md breakpoint ($\geq 768\text{px}$, i.e. tablets/desktops) and up, each article takes up 4 of the 12 columns. Since $12 \div 4 = 3$, exactly three articles fit per row → a 3-column grid on desktop

Why does this work without custom media queries

Because col-md-4 only kicks in at md and above, and col-12 acts as the base/fallback for everything below that, Bootstrap's own internal media queries handle the breakpoint switch. Your HTML just declares the *intent* ("full width until medium screens, then a third of the row"), and Bootstrap's precompiled CSS does the actual responsive switching — no custom @media rules needed on your end.

One useful addition: if you want consistent spacing between cards, you could also add g-3 (or similar gutter class) to the .row for gap control, e.g. `<div class="row g-3">`, though that's a styling refinement rather than a requirement of the 3-column/1-column behavior itself.

5. Scenario 5

You are writing a C program to record daily rainfall data for a month (30 days). You need to store all 30 readings, calculate the monthly average, and then identify which specific days recorded rainfall above that average.

Question: Explain why an array is the correct data structure for this task compared to using 30 separate variables. Describe the two-step logic your program would follow: first to compute the average, then to identify the above-average days — and explain why both steps cannot be done in a single loop.

Answer: Array makes it easy to store 30 different value into the same type of variable that's why array is the correct data structure for this task.

Two step logic:

```
float total = 0;

for (int i = 0; i < 30; i++)
{
    total += rainfall[i];
}

float average = total / 30;
```

Step 2 — Identify above-average days (second loop):

```
c

for (int i = 0; i < 30; i++) {
    if (rainfall[i] > average) {
        printf("Day %d: %.2f mm (above average)\n", i + 1, rainfall[i]);
    }
}
```

Both step can not be done using single loops because to check readings above avg we need total and avg before checking final outcomes in short core problem is dependency.

6. Scenario 6

You are debugging a C program that uses a pointer to traverse a string entered by the user. The program works correctly when the user types a name, but crashes immediately when the input field is left empty (the user just presses Enter).

Question: Explain why the program crashes on an empty string input and describe the check your pointer-based solution must perform before dereferencing the pointer. How does traversing a string using a pointer differ from traversing it using array index notation, and which approach makes the empty-input bug easier to catch?

Answer:

The program crashes because the pointer is dereferenced without checking whether it points to a valid character. For an empty string, the first character is '\0', so the pointer must check for the null terminator before dereferencing. Pointer traversal uses `*p` and `p++`, while array traversal uses `str[i]`. Array indexing is usually easier to spot and debug for empty-input errors.

Practical Coding Task

```
/*
    Task - 1 Grade Band Checker
    Build a console program that accepts a student's percentage score and prints
    their letter
    grade along with a short motivational message.
    1. Accept a float percentage value as user input using scanf().
    2. Assign a grade using if-else if: A (>= 90), B (>= 75), C (>= 60), D (>=
    45), F (below 45).
    3. Print the assigned grade and a one-line message for each band (e.g., 'B –
    Good work! Keep
    pushing.').
    4. Handle invalid input: if the score is outside the range 0-100, print a
    clear error message and
    exit the program gracefully.
*/

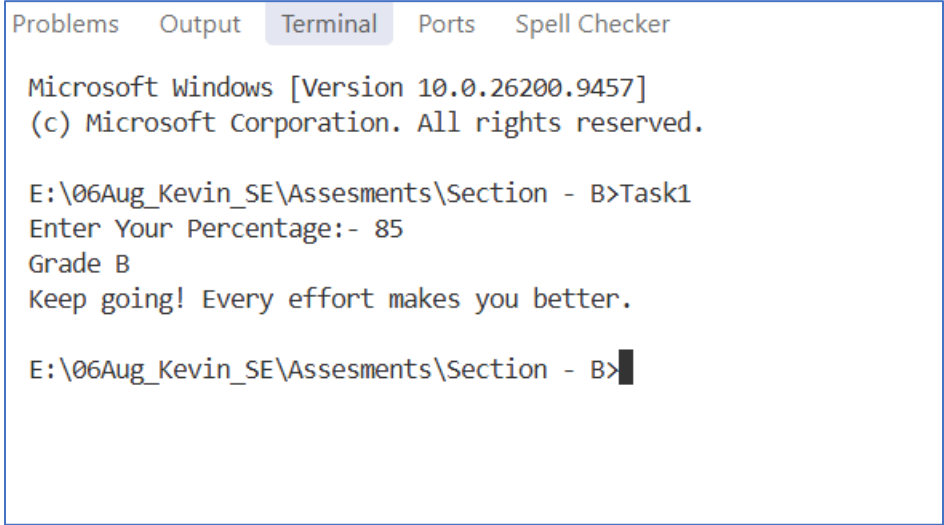
#include<stdio.h>

struct grade
{
    float score;
} ob;

int main()
{
    printf("Enter Your Percentage:- ");
    scanf("%f", &ob.score);

    if(ob.score < 0 || ob.score > 100)
    {
        printf("Invalid i/p\n");
    }
    else
    {
        {
            if(ob.score >= 90)
            {
                printf("Grade A\n");
            }
            else if(ob.score >= 75)
            {
                printf("Grade B\n");
            }
            else if(ob.score >= 60)
```

```
{  
    printf("Grade C\n");  
}  
else if(ob.score >= 45)  
{  
    printf("Grade D\n");  
}  
else  
{  
    printf("FAIL\n");  
}  
  
printf("Keep going! Every effort makes you better.\n");  
}  
return 0;  
}
```

Output:

```
Problems  Output  Terminal  Ports  Spell Checker  
  
Microsoft Windows [Version 10.0.26200.9457]  
(c) Microsoft Corporation. All rights reserved.  
  
E:\06Aug_Kevin_SE\Assesments\Section - B>Task1  
Enter Your Percentage:- 85  
Grade B  
Keep going! Every effort makes you better.  
  
E:\06Aug_Kevin_SE\Assesments\Section - B>
```

```

/*
    Task - 2 Weekly Study Hours Analyzer
    Build a program that records a student's daily study hours for 7 days, stores
    them in an array,
    and generates a performance summary.

    1. Use a for loop to accept 7 float values (one per day) and store them in a
    float array.
    2. Calculate and print the weekly total, daily average, and the day number
    with the highest study
        hours.
    3. Print a simple visual bar for each day: the day number followed by one
    asterisk (*) per hour
        studied, truncated to the nearest integer (e.g., Day 3: ***).
    4. Reject and re-prompt for any day entry that is negative or greater than
    24, before storing it in
        the array.
*/

#include<stdio.h>

struct study_hours
{
    float std_time;
} ob[10];

int main()
{
    int i;
    float total_std_time = 0.0f;
    float avg_std_time;

    for(i=0;i<7;i++)
    {
        do{
            printf("\nEnter your day %d study time:-",i+1);
            scanf("%f",&ob[i].std_time);

            if(ob[i].std_time < 0 || ob[i].std_time>24)
            {
                printf("Invalid entry try again");
            }
        } while (ob[i].std_time < 0 || ob[i].std_time >24);
    }
}

```

```

for(i=0;i<7;i++)
{
    total_std_time += ob[i].std_time;
}

avg_std_time = total_std_time/7;
printf("\nTotal study time: %.1f Hours",total_std_time);
printf("\nAverage study time: %.1f Hours",avg_std_time);

int max = 0;
float max_std_hour = ob[0].std_time;

for(i=1;i<7;i++)
{
    if(ob[i].std_time > max_std_hour)
    {
        max_std_hour= ob[i].std_time;
        max = i;
    }
}
printf("\n\nHighest study hour day: Day%d %.1f hours",max + 1,max_std_hour);

printf("\n-----\n");
printf("    Visual Study Bar    ");
printf("\n-----\n");

for(i=0;i<7;i++)
{
    printf("\nDay %d: ",i+1);
    int hour = (int)(ob[i].std_time);

    for(int j=0;j<hour;j++)
    {
        printf(" * ");
    }
    printf("\n");
}
return 0;
}

```

Output:

```
Problems  Output  Terminal  Ports  Spell Checker

Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

E:\06Aug_Kevin_SE\Assesments\Section - B>gcc Task2.cpp -o Task2.exe

E:\06Aug_Kevin_SE\Assesments\Section - B>Task2

Enter your day 1 study time:-3
Enter your day 2 study time:-4
Enter your day 3 study time:-2
Enter your day 4 study time:-1
Enter your day 5 study time:-3
Enter your day 6 study time:-2
Enter your day 7 study time:-1

Total study time: 16.0 Hours
Average study time: 2.3 Hours

Highest study hour day: Day2 4.0 hours
-----
      Visual Study Bar
-----

Day 1:  * * *
Day 2:  * * * *
Day 3:  * *
Day 4:  *
Day 5:  * * *
Day 6:  * *
Day 7:  *

E:\06Aug_Kevin_SE\Assesments\Section - B>█
```

```
/*
    Task 3: Student Record Manager
    Build a C program using structures and functions to store, display, and
    identify the top
    performer among a group of students.
    1. Define a struct Student with fields: name (char[50]), roll-no (int),
    marks (float), and grade
        (char).
    2. Write a function void assignGrade(struct Student *s) that sets the grade
    field based on the
        marks value using the same bands as Task 1.
    3. In main(), accept data for 3 students, call assignGrade() for each, and
    display all records in a
        formatted table with column headers.
    4. Write a separate function void printTopper(struct Student[], int n) that
    prints the name and
        marks of the student with the highest marks.
*/
```

```
#include<stdio.h>
```

```
struct student {
    char name[50];
    int roll_no;
    float marks;
    char grade;
} st[50];
```

```
void assignGrade(struct student *k)
{
    if(k->marks < 0 || k -> marks > 100)
    {
        printf("Invalid Score!! Your score must in range of 0 to 100\n");
        k->grade = '?';
        return;
    }

    if(k->marks>=90)
    {
        k->grade = 'A';
    }
    else if (k->marks>=75)
    {
        k->grade = 'B';
    }
}
```

```

    else if (k->marks>=60)
    {
        k->grade = 'C';
    }
    else if (k->marks>=45)
    {
        k->grade = 'D';
    }
    else
    {
        k->grade = 'F';
    }
}

/* void printTopper(struct student st[],int n)
{
    if(n<=0)
    {
        printf("|No Student Available in the record|\n");
        return ;
    }

    int max_score = 0;
    int i;
    for(i=0;i<n;i++)
    {
        if(st[i].marks > st[max_score].marks)
        {
            max_score = i;
        }
    }
    printf("\n-----\n");
    printf("                Class Topper                ");
    printf("\n-----\n");

    printf("Roll Number:-%d\n",st[max_score].roll_no);
    printf("Name:-%s\n",st[max_score].name);
    printf("Marks:-%.1f\n",st[max_score].marks);
    printf("Grade:-%c\n",st[max_score].grade);
} */

void printTopper(struct student st[], int n)
{
    if (n <= 0) return;

```

```

int maxIndex = 0;
for (int i = 1; i < n; i++)
{
    if (st[i].marks > st[maxIndex].marks)
    {
        maxIndex = i;
    }
}

printf("\n+-----+\n");
printf("|          CLASS TOPPER          |\n");
printf("+-----+\n");
printf("| %-12s : %-24d |\n", "Roll Number", st[maxIndex].roll_no);
printf("| %-12s : %-24s |\n", "Name", st[maxIndex].name);
printf("| %-12s : %-24.1f |\n", "Marks", st[maxIndex].marks);
printf("| %-12s : %-24c |\n", "Grade", st[maxIndex].grade);
printf("+-----+\n");
}

int main()
{
    int i,n=3;
    for(i=0;i<n;i++)
    {
        printf("\nEnter Roll Number of student %d:-\n",i+1);
        scanf("%d",&st[i].roll_no);

        printf("Enter Name of student %d:-\n",i+1);
        scanf("%s",st[i].name);

        do
        {
            printf("Enter Marks of student %d:-\n",i+1);
            scanf("%f",&st[i].marks);

            if(st[i].marks < 0 || st[i].marks >100)
            {
                printf("\nInvalid Score!! Your score must in range of 0 to
100\n");
            }
        } while (st[i].marks <0 || st[i].marks>100);
        assignGrade(&st[i]);
    }
}

```

```
        printf("\n%-12s %-15s %-15s %-5s" , "Roll
Number", "Name", "Marks", "Grade\n");
        printf("-----\n");
        for(i=0;i<n;i++)
        {
            printf("%-12d %-15s %-15.1f %-5c\n",
                st[i].roll_no,
                st[i].name,
                st[i].marks,
                st[i].grade);
        }
        printf("-----\n");

        printTopper(st,n);
        return 0;
    }
```

Output:

```
Problems  Output  Terminal  Ports  Spell Checker

E:\06Aug_Kevin_SE\Assesments\Section - B>Task3

Enter Roll Number of student 1:-
100
Enter Name of student 1:-
Kevin
Enter Marks of student 1:-
77

Enter Roll Number of student 2:-
101
Enter Name of student 2:-
Shubman
Enter Marks of student 2:-
70

Enter Roll Number of student 3:-
103
Enter Name of student 3:-
Trisha
Enter Marks of student 3:-
90

Roll Number  Name          Marks      Grade
-----
100          Kevin           77.0       B
101          Shubman          70.0       C
103          Trisha           90.0       A
-----

+-----+
|          CLASS TOPPER          |
+-----+
| Roll Number : 103              |
| Name       : Trisha            |
| Marks      : 90.0              |
| Grade      : A                 |
+-----+
```

E:\06Aug_Kevin_SE\Assesments\Section - B>█

```

/*
    Task 4: Personal Expense Logger
    Build a menu-driven C program that lets a user log and view personal
    daily expenses, and
    saves all records to a text file on exit.
    1. Define a struct Expense with fields: category (char[30]) and amount
    (float). Allocate an array
        to store up to 10 Expense entries.
    2. Present a menu with 3 options: (1) Add Expense, (2) View All
    Expenses, (3) Save & Exit. Loop until
        the user selects option 3.
    3. In the View option, display all logged expenses in a formatted list
    and print the running total at
        the bottom.
    4. On exit (option 3), write all expense records to a file named
    expenses.txt using fprintf(), with
        one record per line in the format: category,amount.
*/

```

```
#include<stdio.h>
```

```
struct expense
```

```
{
    char category[30];
    float amount;
} ex[10];
```

```
int main()
```

```
{
    int n=10;
    int choice;
    float total_expense = 0;
```

```
    printf("\n<<<=====>>>\n");
    printf("|      Welcome to Personal Expense Logger      |");
    printf("\n<<<=====>>>\n");
    // printf("\n");
```

```
    while(1)
```

```
    {
        printf("\n<<<=====>>>\n");
        printf("|          Press 1 for Add Expense          |\n");
        printf("|          Press 2 for View Expense          |\n");
        printf("|          Press 3 for Save & Exit          |");
        printf("\n<<<=====>>>\n");
```

```

printf("\nEnter Your Choice >>> ");
scanf("%d",&choice);

switch (choice)
{
case 1:
    printf("\n<<<=====>>>\n");
    for(int i=0;i<n;i++)
    {
        printf("Enter Your Expense Category %d:",i+1);
        scanf("%s",&ex[i].category);

        printf("Enter Your Expense amount of Category %d:",i+1);
        scanf("%f",&ex[i].amount);
        printf("\n\n");
    }
    printf("<<<=====>>>\n");
    break;
case 2:
    for(int i=0;i<n;i++)
    {
        total_expense +=ex[i].amount;
    }
    printf("\n+-----+-----+\n");
    printf("| %-20s | %-10s |\n", "Expense Category", "Amount");
    printf("+-----+-----+\n");

    for(int i=0;i<n;i++)
    {
        printf("| %-20s | %-10.2f |\n", ex[i].category, ex[i].amount);
    }
    printf("+-----+-----+\n");
    printf("| %-20s | %-10.2f |\n","Total Expense",total_expense);
    printf("+-----+-----+\n");
    break;
case 3:
    {
        FILE *ob;
        ob = fopen("expense.txt","w");
        // printf("FILE CREATED");

        fprintf(ob, "\n+-----+-----+\n");
        fprintf(ob, "| %-20s | %-10s |\n", "Expense Category", "Amount");
    }
}

```

```
    fprintf(ob, "+-----+-----+\n");

    for(int i=0;i<n;i++)
    {
        fprintf(ob, "| %-20s | %-10.2f |\n", ex[i].category, ex[i].amount);
    }
    fprintf(ob, "+-----+-----+\n");
    fprintf(ob, "| %-20s | %-10.2f |\n", "Total Expense", total_expense);
    fprintf(ob, "+-----+-----+\n");

}
break;

default:
    printf("Error! Invalid Choice");
    break;
}

if(choice == 3)
{
    break;
}
}
return 0;
}
```

Output:

```

Problems  Output  Terminal  Ports  Spell Checker

Enter Your Expense Category 8:Cloths
Enter Your Expense amount of Category 8:1000

Enter Your Expense Category 9:Recharge
Enter Your Expense amount of Category 9:999

Enter Your Expense Category 10:Giftcard
Enter Your Expense amount of Category 10:10

<<<=====>>>

<<<=====>>>
|               Press 1 for Add Expense               |
|               Press 2 for View Expense               |
|               Press 3 for Save & Exit                 |
|               |               |                       |
<<<=====>>>

Enter Your Choice >>> 2

+-----+-----+
| Expense Category | Amount |
+-----+-----+
| Petrol           | 250.00 |
| Milk             | 66.00  |
| Travel           | 500.00 |
| Snacks           | 100.00 |
| MovieShow        | 300.00 |
| Grocery          | 600.00 |
| Veg              | 400.00 |
| Cloths           | 1000.00|
| Recharge         | 999.00 |
| Giftcard         | 10.00  |
+-----+-----+
| Total Expense   | 4225.00|
+-----+-----+

<<<=====>>>
|               Press 1 for Add Expense               |
|               Press 2 for View Expense               |
|               Press 3 for Save & Exit                 |
|               |               |                       |
<<<=====>>>

Enter Your Choice >>> 3

E:\06Aug_Kevin_SE\Assesments\Section - B>

```

```
/*  
    Mini Project:  Student Productivity Tracker  
    Objective:  
  
        Build a console-based Student Productivity Tracker that logs daily study  
hours across subjects  
        for a full week, combining arrays, structures, functions, and file handling  
from Module 3 into a  
        single working program.  
Your project must:  
  
        The program must be menu-driven with at least 3 options: (1) Log Today's  
Study Hours, (2)  
        View Weekly Report, (3) Save & Exit.  
  
        Define a struct StudyLog { char subject[40]; float hours[7]; } and create an  
array of at least 3  
        subject records.  
  
        Write a function that calculates and displays the weekly total hours and  
daily average for  
        each subject.  
  
        Display a simple text-based progress chart: for each subject, print one  
filled dot (•) per hour  
        studied that day (truncate to nearest integer).  
  
        On exit, save all records to a file named productivity_log.txt using  
fprintf(), with each subject's  
        name and 7 daily values written as a single comma-separated line.  
*/
```

```

#include<stdio.h>

int i,j;

struct StudyLog
{
    char subject[50];
    float hours[7];
} st[3];

void weekly_total()
{
    float total_hours = 0;
    printf("<<<===== Weekly Summary =====>>>\n");
    for(i=0;i<3;i++)
    {
        for(j=0;j<7;j++)
        {
            total_hours += st[i].hours[j];
        }
    }

    float avg = total_hours/7.0;

    printf("Subject : %-15s | Total Hours : %-6.2f | Average : %-5.2f",
        st[i].subject,total_hours,avg);

    printf("<<<-----\n>>>");
}

void progress_chart()
{
    printf("\n===== PROGRESS CHART =====\n");
    int k;

    for(i=0;i<3;i++)
    {
        printf("\n|Subjects: %s|\n",st[i].subject);
        for(j=0;j<7;j++)
        {
            int chart = (int)(st[i].hours[j]);
            printf("Day %d >>>",j+1);

            for(k=0;k<chart;k++)
            {

```

```

        printf(" . ");
    }
    printf(" (%.1f hrs)\n", st[i].hours[j]);
}
}
printf("=====\n");
}

int main()
{
    int choice;
    // int i,j;

    for(i=0;i<3;i++)
    {
        printf("Enter the name of subject %d",i+1);
        scanf("%s",st[i].subject);
    }

    while (1)
    {
        printf("\n<<<=====>>>\n");
        printf("| Welcome to Student Productivity Tracker |");
        printf("\n<<<=====>>>\n");
        printf("| 1. Log Study Hours | \n");
        printf("| 2. View Weekly Report & Progress Chart | \n");
        printf("| 3. Save & Exit | \n");
        printf("<<<=====>>>\n");
        printf("Enter Your Choice >>> ");
        scanf("%d",&choice);

        switch (choice)
        {
            case 1:
                for(i=0;i<3;i++)
                {
                    printf("\nLogging Hours: %s Subject\n",st[i].subject);
                    {
                        for(j=0;j<7;j++)
                        {
                            printf("Day % d Hours >>>",j+1);
                            scanf("%f",&st[i].hours[j]);
                        }
                    }
                }
            }
        }
    }
}

```

```
        break;

    case 2:
        weekly_total();
        progress_chart();
        break;

    case 3:

FILE *ob;
ob = fopen("Productivity_log.txt", "a");

    for(i=0;i<3;i++)
    {
        fprintf(ob,"%s",st[i].subject);
        for(j=0;j<7;j++)
        {
            fprintf(ob,",%.2f hh",st[i].hours[j]);
        }
        fprintf(ob,"\n");
    }

    fclose(ob);
    return 0;
    break;

    default:
        printf("\n Error ! Kindly go with 1 or 2 or 3 \n");
        break;
    }

    if (choice == 3)
    {
        break;
    }
}

return 0;
}
```

Output: -

```
Problems  Output  Terminal  Ports  Spell Checker

Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

E:\06Aug_Kevin_SE\Assesments\Section - C>mini_projectt
Enter the name of subject 1Java
Enter the name of subject 2Python
Enter the name of subject 3Css

<<<=====>>>
| Welcome to Student Productivity Tracker |
<<<=====>>>
| 1. Log Study Hours |
| 2. View Weekly Report & Progress Chart |
| 3. Save & Exit |
<<<=====>>>
Enter Your Choice >>> 1

Logging Hours: Java Subject
Day 1 Hours >>>2
Day 2 Hours >>>3
Day 3 Hours >>>4
Day 4 Hours >>>5
Day 5 Hours >>>4
Day 6 Hours >>>3
Day 7 Hours >>>2

Logging Hours: Python Subject
Day 1 Hours >>>2
Day 2 Hours >>>3
Day 3 Hours >>>4
Day 4 Hours >>>5
Day 5 Hours >>>4
Day 6 Hours >>>1
Day 7 Hours >>>2

Logging Hours: Css Subject
Day 1 Hours >>>1
Day 2 Hours >>>2
Day 3 Hours >>>3
Day 4 Hours >>>5
Day 5 Hours >>>4
Day 6 Hours >>>3
Day 7 Hours >>>2

<<<=====>>>
| Welcome to Student Productivity Tracker |
```

Problems Output **Terminal** Ports Spell Checker

Day 7 Hours >>>2

```
<<<=====>>>
```

```
| Welcome to Student Productivity Tracker |
```

```
<<<=====>>>
```

```
| 1. Log Study Hours |
```

```
| 2. View Weekly Report & Progress Chart |
```

```
| 3. Save & Exit |
```

```
<<<=====>>>
```

Enter Your Choice >>> 2

```
<<<===== Weekly Summary =====>>>
```

```
Subject : | Total Hours : 64.00 | Average : 9.14 <<<-----
```

```
>>>
```

```
===== PROGRESS CHART =====
```

```
|Subjects: Java|
```

```
Day 1 >>> . . (2.0 hrs)
```

```
Day 2 >>> . . . (3.0 hrs)
```

```
Day 3 >>> . . . (4.0 hrs)
```

```
Day 4 >>> . . . . (5.0 hrs)
```

```
Day 5 >>> . . . . (4.0 hrs)
```

```
Day 6 >>> . . . (3.0 hrs)
```

```
Day 7 >>> . . (2.0 hrs)
```

```
|Subjects: Python|
```

```
Day 1 >>> . . (2.0 hrs)
```

```
Day 2 >>> . . . (3.0 hrs)
```

```
Day 3 >>> . . . . (4.0 hrs)
```

```
Day 4 >>> . . . . (5.0 hrs)
```

```
Day 5 >>> . . . . (4.0 hrs)
```

```
Day 6 >>> . (1.0 hrs)
```

```
Day 7 >>> . . (2.0 hrs)
```

```
|Subjects: Css|
```

```
Day 1 >>> . (1.0 hrs)
```

```
Day 2 >>> . . (2.0 hrs)
```

```
Day 3 >>> . . . (3.0 hrs)
```

```
Day 4 >>> . . . . (5.0 hrs)
```

```
Day 5 >>> . . . . (4.0 hrs)
```

```
Day 6 >>> . . . (3.0 hrs)
```

```
Day 7 >>> . . (2.0 hrs)
```

```
=====
```

```
<<<=====>>>
```

```
| Welcome to Student Productivity Tracker |
```

```
<<<=====>>>
```

```
Section - C > Productivity_log.txt
You, 1 minute ago | author (You)
1 java,2.00,2.00,2.00,4.00,4.00,4.00,3.00
2 python,1.00,2.00,3.00,4.00,5.00,6.00,3.00
3 css,4.00,5.00,3.00,2.00,4.00,2.00,1.00 You, 8 minutes ago • initial commit
4 java,1.00 hh,2.00 hh,3.00 hh,4.00 hh,3.00 hh,2.00 hh,1.00 hh
5 python,3.00 hh,4.00 hh,2.00 hh,5.00 hh,1.00 hh,6.00 hh,7.00 hh
6 css,1.00 hh,2.00 hh,4.00 hh,3.00 hh,2.00 hh,1.00 hh,3.00 hh
7 Java,2.00 hh,3.00 hh,4.00 hh,5.00 hh,4.00 hh,3.00 hh,2.00 hh
8 Python,2.00 hh,3.00 hh,4.00 hh,5.00 hh,4.00 hh,1.00 hh,2.00 hh
9 Css,1.00 hh,2.00 hh,3.00 hh,5.00 hh,4.00 hh,3.00 hh,2.00 hh
10
```

```
/*
    STEP 1    ·    BUILD WITH AI

    Use an AI tool of your choice (ChatGPT, Claude, GitHub Copilot, etc.) to help
    you write a C program that:
    1.  Accepts exactly 10 integers from the user using a loop and stores them in
    an array.
    2.  Finds and displays the maximum value, minimum value, and arithmetic mean
    (displayed as a
        float with 2 decimal places).
    3.  Sorts the array in ascending order using any sorting method and displays
    the sorted list.
    4.  Prints whether the mean is closer to the minimum, closer to the maximum,
    or exactly midway
        between them
*/

/*
    Prompt Given to ai (ChatGpt)

    Write a c code that accept exactly 10 integers from the user using for loop
    and store
    them in an array and then find maximum and minimum value and
    arith-mic mean displayed as float with 2 decimal places
    and sort the array in ascending order using any sorting method and
    displays the sorted list. & Prints whether the mean is closer to the minimum,
    closer to the maximum, or exactly midway between them.
*/

#include <stdio.h>

int main()
{
    int arr[10];
    int i, j, temp;
    int max, min;
    float mean;

    // Accept exactly 10 integers
    printf("Enter 10 integers:\n");

    for(i = 0; i < 10; i++)
    {
        printf("Enter number %d: ", i + 1);
        scanf("%d", &arr[i]);
    }
}
```

```
}

// Assume first element is maximum and minimum
max = arr[0];
min = arr[0];

// Find maximum and minimum
for(i = 1; i < 10; i++)
{
    if(arr[i] > max)
    {
        max = arr[i];
    }

    if(arr[i] < min)
    {
        min = arr[i];
    }
}

// Calculate arithmetic mean
int sum = 0;

for(i = 0; i < 10; i++)
{
    sum = sum + arr[i];
}

mean = (float)sum / 10;

// Sort array in ascending order
for(i = 0; i < 9; i++)
{
    for(j = i + 1; j < 10; j++)
    {
        if(arr[i] > arr[j])
        {
            temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
}

// Display results
```

```
printf("\nMaximum = %d\n", max);
printf("Minimum = %d\n", min);
printf("Arithmetic Mean = %.2f\n", mean);

// Display sorted array
printf("Sorted array in ascending order:\n");

for(i = 0; i < 10; i++)
{
    printf("%d ", arr[i]);
}

printf("\n");

// Check where mean lies
float distanceFromMin = mean - min;
float distanceFromMax = max - mean;

if(distanceFromMin < distanceFromMax)
{
    printf("Mean is closer to the minimum.\n");
}
else if(distanceFromMin > distanceFromMax)
{
    printf("Mean is closer to the maximum.\n");
}
else
{
    printf("Mean is exactly midway between the minimum and maximum.\n");
}
return 0;
}
```

Output:

```
Problems  Output  Terminal  Ports  Spell Checker

Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

E:\06Aug_Kevin_SE\Assesments\Section - D>gcc ai_code.cpp -o ai_code.exe

E:\06Aug_Kevin_SE\Assesments\Section - D>ai_code
Enter 10 integers:
Enter number 1: 2
Enter number 2: 3
Enter number 3: 4
Enter number 4: 5
Enter number 5: 5
Enter number 6: 4
Enter number 7: 3
Enter number 8: 2
Enter number 9: 1
Enter number 10: 2

Maximum = 5
Minimum = 1
Arithmetic Mean = 3.10
Sorted array in ascending order:
1 2 2 2 3 3 4 4 5 5
Mean is closer to the maximum.

E:\06Aug_Kevin_SE\Assesments\Section - D>ai_code
Enter 10 integers:
Enter number 1: -10
Enter number 2: -20
Enter number 3: 30
Enter number 4: 40
Enter number 5: 50
Enter number 6: -100
Enter number 7: 50
Enter number 8: 40
Enter number 9: 30
Enter number 10: 12

Maximum = 50
Minimum = -100
Arithmetic Mean = 12.20
Sorted array in ascending order:
-100 -20 -10 12 30 30 40 40 50 50
Mean is closer to the maximum.

E:\06Aug_Kevin_SE\Assesments\Section - D>
```

```
#include<stdio.h>

// gcc my_code.cpp -o my_code
int main()
{
    int number[10];
    int i,j;
    int min,max,temp;

    for(i=0;i<10;i++)
    {
        printf("Enter any 10 int Numbers:-");
        scanf("%d",&number[i]);
    }

    min = number[0];
    max = number[0];

    for(i=0;i<10;i++)
    {
        if(number[i]>max)
        {
            max = number[i];
        }

        if(number[i]<min)
        {
            min = number[i];
        }
    }

    printf("Max Value:- %d\n",max);
    printf("Min Value:- %d\n",min);

    // calculating mean

    int sum = 0;
    for(i=0;i<10;i++)
    {
        sum += number[i];
    }

    printf("Sum of 10 Integer number:%d\n",sum);
```

```
float mean = (float)sum/10;

printf("Arithmetic Mean: %.2f\n", mean);

// sorting array

for(i=0; i<10; i++)
{
    for(j=i+1; j<10; j++)
    {
        if (number[i] > number[j])
        {
            temp = number[i];
            number[i] = number[j];
            number[j] = temp;
        }
    }
}

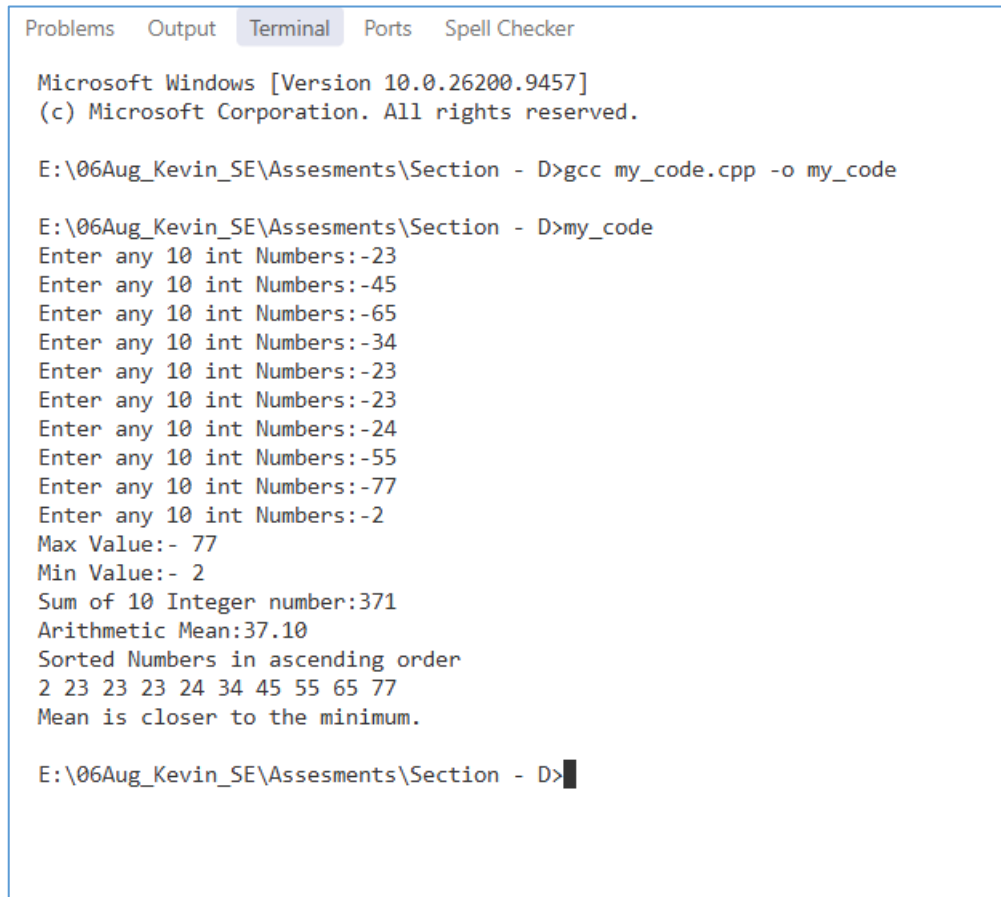
printf("Sorted Numbers in ascending order \n");

for(i=0; i<10; i++)
{
    printf("%d ", number[i]);
}
printf("\n");

float distanceFromMin = mean - min;
float distanceFromMax = max - mean;

if(distanceFromMin < distanceFromMax)
{
    printf("Mean is closer to the minimum.\n");
}
else if(distanceFromMin > distanceFromMax)
{
    printf("Mean is closer to the maximum.\n");
}
else
{
    printf("Mean is exactly midway between the minimum and maximum.\n");
}
return 0;
}
```

My_code.cpp

Output:

```
Problems Output Terminal Ports Spell Checker

Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

E:\06Aug_Kevin_SE\Assesments\Section - D>gcc my_code.cpp -o my_code

E:\06Aug_Kevin_SE\Assesments\Section - D>my_code
Enter any 10 int Numbers:-23
Enter any 10 int Numbers:-45
Enter any 10 int Numbers:-65
Enter any 10 int Numbers:-34
Enter any 10 int Numbers:-23
Enter any 10 int Numbers:-23
Enter any 10 int Numbers:-24
Enter any 10 int Numbers:-55
Enter any 10 int Numbers:-77
Enter any 10 int Numbers:-2
Max Value:- 77
Min Value:- 2
Sum of 10 Integer number:371
Arithmetic Mean:37.10
Sorted Numbers in ascending order
2 23 23 23 24 34 45 55 65 77
Mean is closer to the minimum.

E:\06Aug_Kevin_SE\Assesments\Section - D>
```

- Ai version needed changes in variable naming and some code of structure.
- I change some name of variable and done some parts of code at my own.